Name: Patel Ansh V.
Enrollment No.: 12402080601005

Branch: IT-A (Sem 5)

Batch : 1A8

Course Name : PROGRAMMING WITH PYTHON

Course Code : 202044504

INDEX

SR NO	TITLE	DATE	SINGATURE
1	ASSIGNMENT - 1	30 / Sep / 2026	

Q1. Campus Merit Analyzer using Compound Data Structures

Code :

```
def main():
    n, k, m = map(int, input().split())

    semester_students = {}
    students = []

    for _ in range(n):
        data = input().split()

        enrollment = data[0]
        name = data[1]
        semester = int(data[2])
        cpi = float(data[3])
        marks = tuple(map(int, data[4:4 + m]))
        avg_marks = sum(marks) / m

        student = (enrollment, name, semester, cpi, marks, avg_marks)
        students.append(student)

        if semester not in semester_students:
            semester_students[semester] = []

        semester_students[semester].append(student)

    for semester in sorted(semester_students.keys()):
        ranked_students = sorted(
            semester_students[semester],
            key=lambda x: (-x[3], -x[5], x[0])
        )

        top_k = ranked_students[:k]

        print(
            f"Semester {semester}:",
            *[student[0] for student in top_k]
        )

    for subject_index in range(m):
        highest_mark = -1
        toppers = []

        for student in students:
            enrollment = student[0]
            marks = student[4]
```

```
        mark = marks[subject_index]

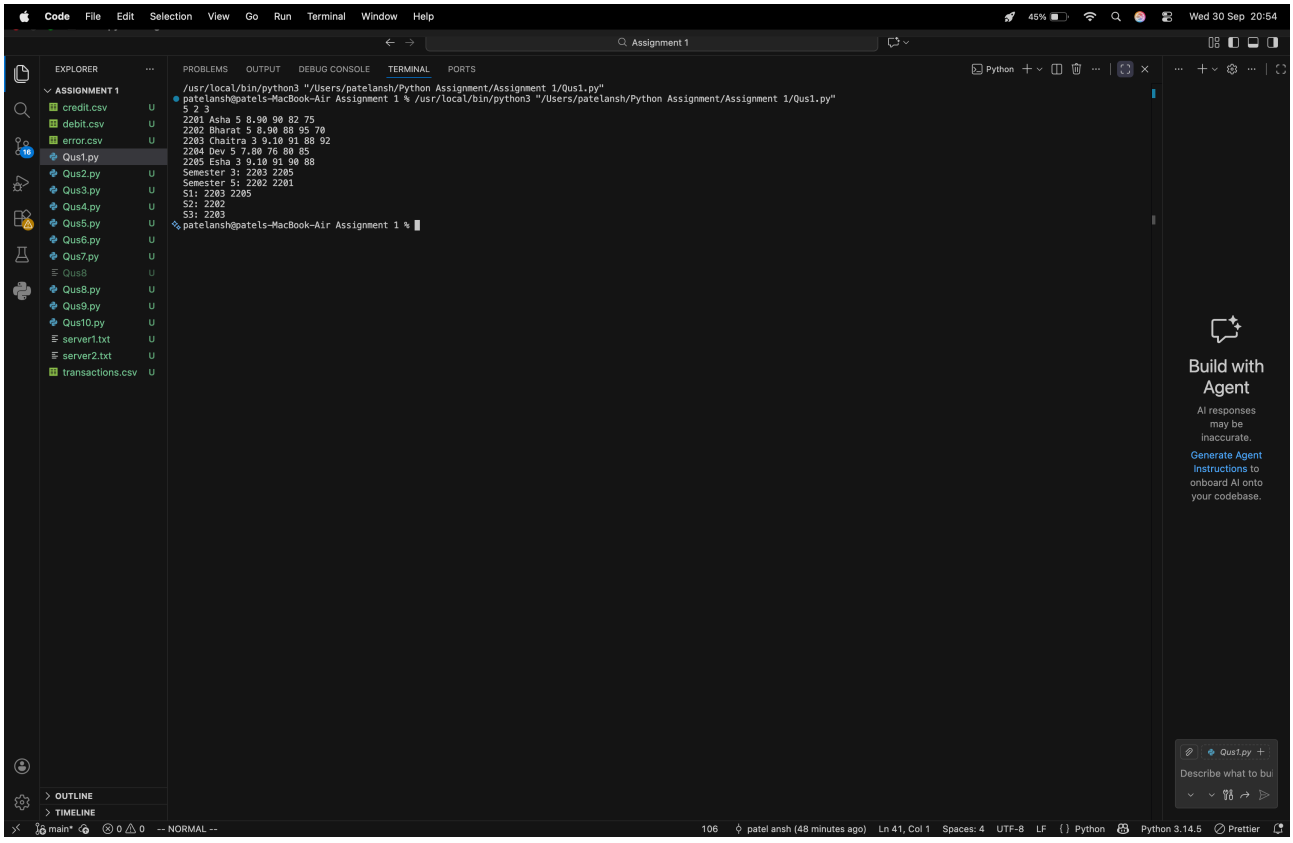
        if mark > highest_mark:
            highest_mark = mark
            toppers = [enrollment]

        elif mark == highest_mark:
            toppers.append(enrollment)

    toppers.sort()
    print(f"S{subject_index + 1}:", *toppers)


if __name__ == "__main__":
    main()
```

Output :



Q2. Optimized Password Audit with Pattern Constraints

Code :

```
import re

def main():
    b = int(input())
    banned_words = [input().strip().lower() for _ in range(b)]

    n = int(input())

    for i in range(1, n + 1):
        password = input().strip()

        if len(password) < 6 or len(password) > 12:
            print(f"{i}: WEAK_LENGTH")
            continue

        lower = re.search(r"[a-z]", password)
        upper = re.search(r"[A-Z]", password)
        digit = re.search(r"\d", password)
        special = re.search(r"[$#@]", password)

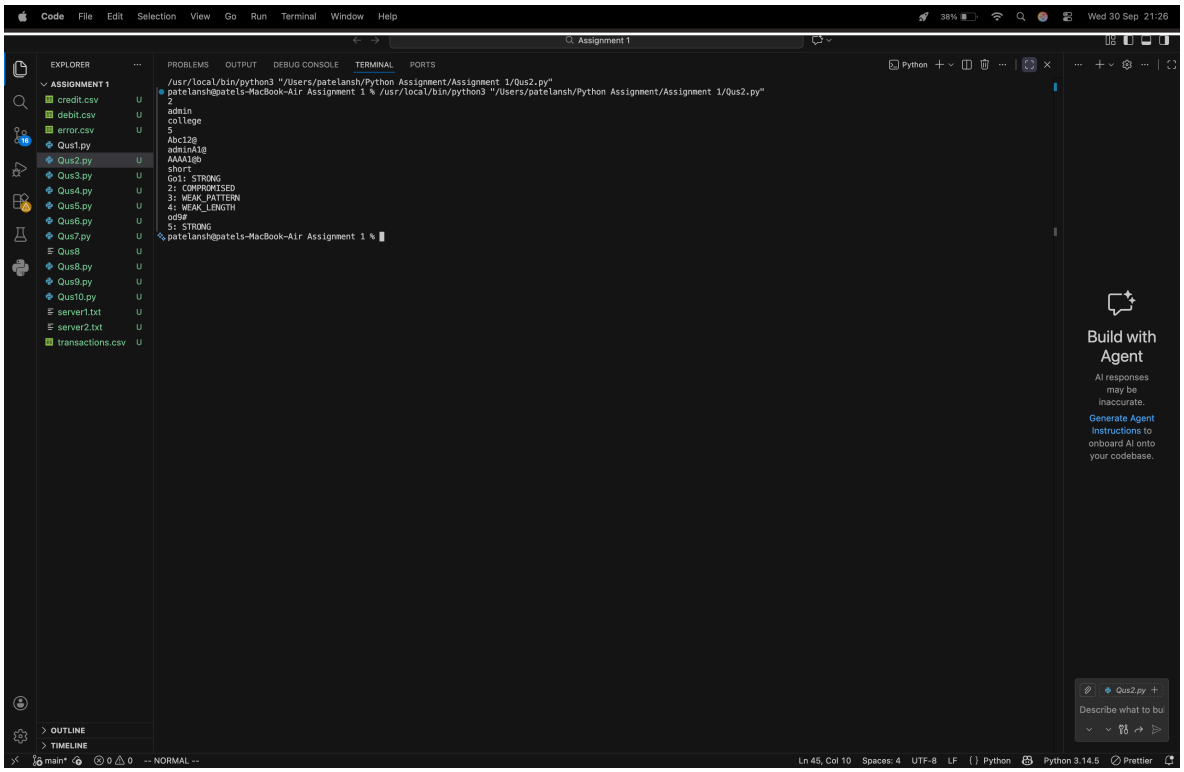
        if not (lower and upper and digit and special):
            print(f"{i}: WEAK_PATTERN")
            continue
        password_lower = password.lower()
        compromised = False
        for word in banned_words:
            if word in password_lower:
                compromised = True
                break

        if compromised:
            print(f"{i}: COMPROMISED")
            continue
        if re.search(r"(.)\1\1\1", password):
            print(f"{i}: WEAK_PATTERN")
            continue

        print(f"{i}: STRONG")

if __name__ == "__main__":
    main()
```

Output :



Q3. Recursive Expression Engine with Memoization

Code :

```
import re
import sys

variables = {}
memo = {}
visiting = set()

def tokenize(expression):
    return re.findall(r'\d+|[A-Za-z_]\w*|[()+\-*]', expression)

def evaluate_variable(name):
    if name in memo:
        return memo[name]

    if name in visiting:
```

```
            raise ValueError("CYCLE")

        if name not in variables:
            raise ValueError("INVALID")

        visiting.add(name)

        try:
            tokens = tokenize(variables[name])
            value, pos = parse_expression(tokens, 0)

            if pos != len(tokens):
                raise ValueError("INVALID")

            memo[name] = value
            return value

        finally:
            visiting.remove(name)

    def parse_expression(tokens, pos):
        value, pos = parse_term(tokens, pos)

        while pos < len(tokens) and tokens[pos] in ('+', '-'):
            operator = tokens[pos]
            right, pos = parse_term(tokens, pos + 1)

            if operator == '+':
                value += right
            else:
                value -= right

        return value, pos

    def parse_term(tokens, pos):
        value, pos = parse_factor(tokens, pos)

        while pos < len(tokens) and tokens[pos] == '*':
            right, pos = parse_factor(tokens, pos + 1)
            value *= right

        return value, pos

    def parse_factor(tokens, pos):
        if pos >= len(tokens):
            raise ValueError("INVALID")

        token = tokens[pos]
```

```
            if token == '(':
                value, pos = parse_expression(tokens, pos + 1)

                if pos >= len(tokens) or tokens[pos] != ')':
                    raise ValueError("INVALID")

                return value, pos + 1

            if token.isdigit():
                return int(token), pos + 1

            if re.fullmatch(r'[A-Za-z_]\w*', token):
                return evaluate_variable(token), pos + 1

            raise ValueError("INVALID")

        def main():
            v = int(input())

            for _ in range(v):
                line = input()

                if '=' not in line:
                    print("INVALID")
                    return

                name, expression = line.split('=', 1)
                name = name.strip()
                expression = expression.strip()

                if not re.fullmatch(r'[A-Za-z_]\w*', name):
                    print("INVALID")
                    return

                variables[name] = expression

            expression = input()

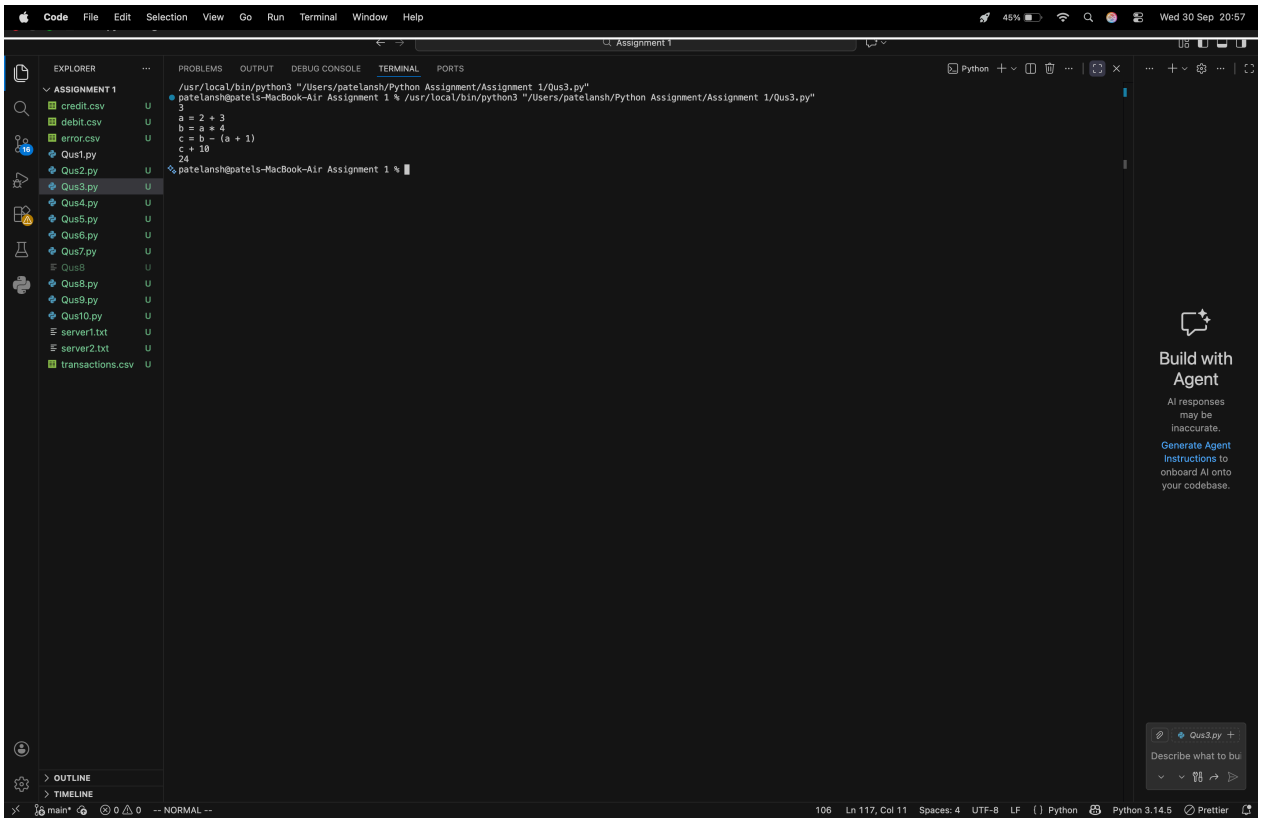
            try:
                tokens = tokenize(expression)
                value, pos = parse_expression(tokens, 0)

                if pos != len(tokens):
                    print("INVALID")
                else:
                    print(value)

            except ValueError as error:
                print(str(error))
```

```
if __name__ == "__main__":
    main()
```

Output :



Q4. Exception-Safe CSV Transaction Splitter

Code :

```
import csv
from datetime import datetime
from collections import defaultdict

def main():
    file_path = input().strip()

    balances = defaultdict(float)
    errors = []
    credits = []
    debits = []

    with open(file_path, "r", newline="") as file:
        reader = csv.DictReader(file)

        for row in reader:
```

```
                try:
                    tid = row["tid"]
            acc = row["acc"]
            transaction_type = row["type"].upper()
            amount = float(row["amount"])
            timestamp = row["time"]

            if not tid or not acc:
                raise ValueError("Missing transaction or account ID")

            if transaction_type not in ("CREDIT", "DEBIT"):
                raise ValueError("Invalid transaction type")

            if amount <= 0:
                raise ValueError("Amount must be greater than 0")

            datetime.strptime(timestamp, "%Y-%m-%dT%H:%M:%S")

            if transaction_type == "CREDIT":
                credits.append(row)
                balances[acc] += amount
            else:
                debits.append(row)
                balances[acc] -= amount

        except Exception as e:
            row["reason"] = str(e)
            errors.append(row)

with open("credit.csv", "w", newline="") as file:
    writer = csv.DictWriter(
        file,
        fieldnames=["tid", "acc", "type", "amount", "time"]
    )
    writer.writeheader()
    writer.writerows(credits)

with open("debit.csv", "w", newline="") as file:
    writer = csv.DictWriter(
        file,
        fieldnames=["tid", "acc", "type", "amount", "time"]
    )
    writer.writeheader()
    writer.writerows(debits)

with open("error.csv", "w", newline="") as file:
    writer = csv.DictWriter(
        file,
        fieldnames=["tid", "acc", "type", "amount", "time", "reason"]
    )
    writer.writeheader()
```

```
        writer.writerows(errors)

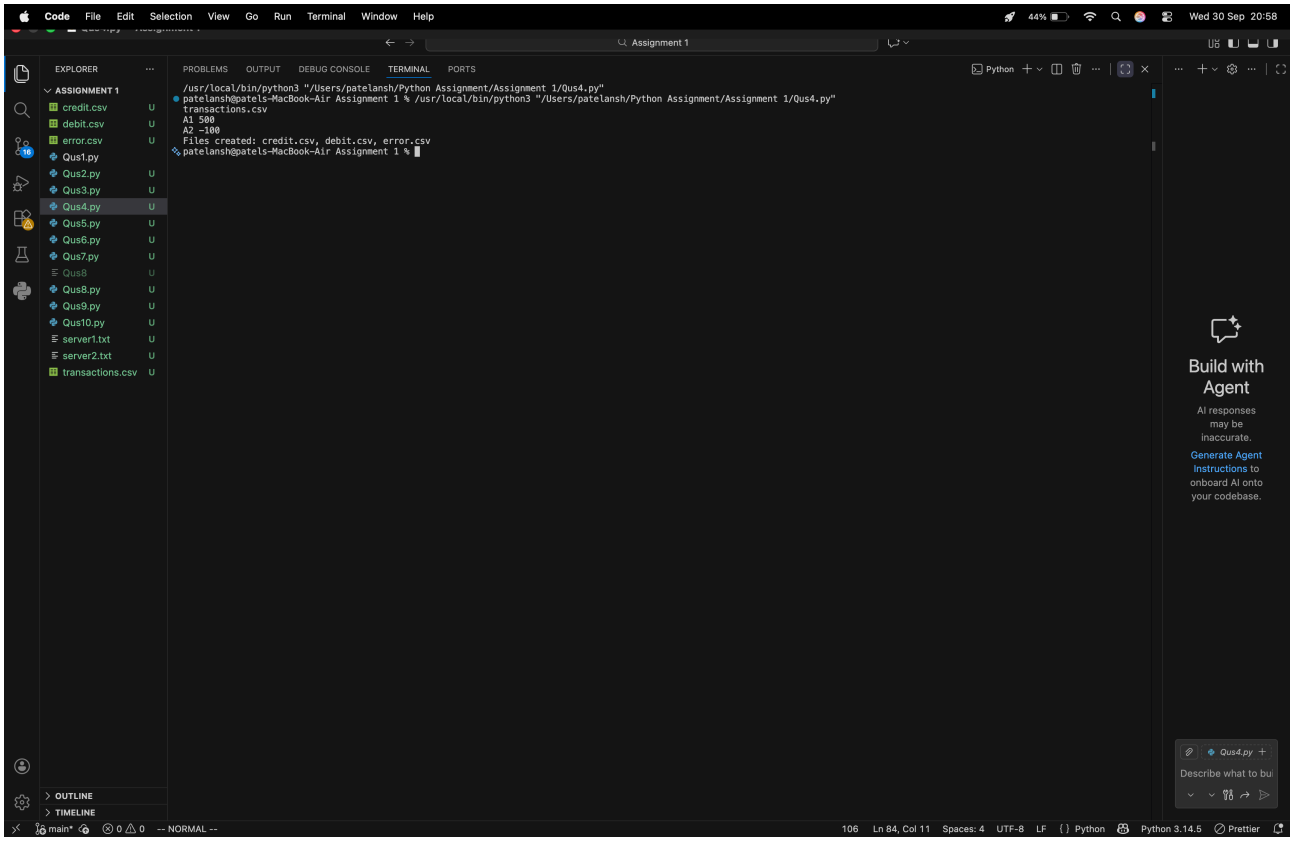
    result = sorted(
        balances.items(),
        key=lambda x: (-abs(x[1]), x[0])
    )

    for account, balance in result:
        if balance.is_integer():
            balance = int(balance)
        print(account, balance)

    print("Files created: credit.csv, debit.csv, error.csv")


if __name__ == "__main__":
    main()
```

Output :



Q5. Object-Oriented Bank Settlement System

Code :

```
class InsufficientBalanceError(Exception):
    pass


class AccountNotFoundError(Exception):
    pass


class Account:
    def __init__(self, account_id, balance):
        self.account_id = account_id
        self._balance = balance
        self._history = []

    @property
    def balance(self):
        return self._balance

    @property
    def history(self):
        return self._history.copy()

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Invalid amount")

        self._balance += amount
        self._history.append(("DEPOSIT", amount))

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("Invalid amount")

        if amount > self._balance:
            raise InsufficientBalanceError("Insufficient balance")

        self._balance -= amount
        self._history.append(("WITHDRAW", amount))


class Transaction:
    def __init__(self, operation, account_from=None, account_to=None, amount=0):
        self.operation = operation
```

```
        self.account_from = account_from
        self.account_to = account_to
        self.amount = amount


class Bank:
    def __init__(self):
        self.accounts = {}
        self.batch_number = 0
        self.failed_batches = []

    def add_account(self, account_id, balance):
        self.accounts[account_id] = Account(account_id, balance)

    def get_account(self, account_id):
        if account_id not in self.accounts:
            raise AccountNotFoundError("Account not found")
        return self.accounts[account_id]

    def deposit(self, account_id, amount):
        self.get_account(account_id).deposit(amount)

    def withdraw(self, account_id, amount):
        self.get_account(account_id).withdraw(amount)

    def transfer(self, from_id, to_id, amount):
        sender = self.get_account(from_id)
        receiver = self.get_account(to_id)

        sender.withdraw(amount)
        receiver.deposit(amount)

    def execute(self, operation):
        parts = operation.split()

        if parts[0] == "DEPOSIT":
            self.deposit(parts[1], int(parts[2]))

        elif parts[0] == "WITHDRAW":
            self.withdraw(parts[1], int(parts[2]))

        elif parts[0] == "TRANSFER":
            self.transfer(parts[1], parts[2], int(parts[3]))

    def execute_batch(self, operations):
        self.batch_number += 1

        old_balances = {
            account_id: account.balance
```

```
            for account_id, account in self.accounts.items()
        }

        old_history = {
            account_id: account.history
            for account_id, account in self.accounts.items()
        }

        try:
            for operation in operations:
                self.execute(operation)

        except Exception:
            for account_id, account in self.accounts.items():
                account._balance = old_balances[account_id]
                account._history = old_history[account_id]

            self.failed_batches.append(self.batch_number)
            print(f"FAILED {self.batch_number}")


def main():
    bank = Bank()

    n = int(input())

    for _ in range(n):
        account_id, balance = input().split()
        bank.add_account(account_id, int(balance))

    q = int(input())

    batch_operations = []
    in_batch = False

    for _ in range(q):
        operation = input().strip()

        if operation == "BATCH_BEGIN":
            in_batch = True
            batch_operations = []

        elif operation == "BATCH_END":
            bank.execute_batch(batch_operations)
            in_batch = False

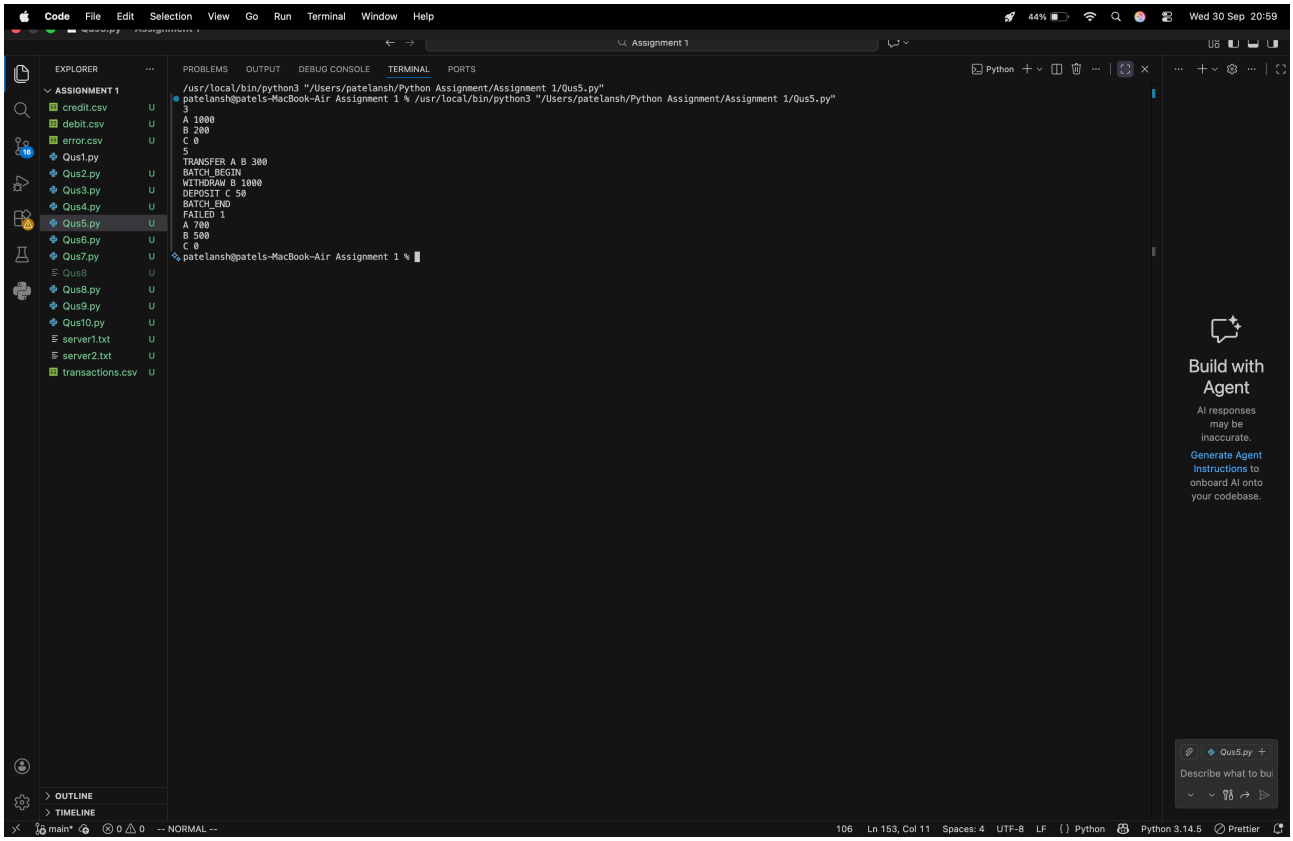
        elif in_batch:
            batch_operations.append(operation)
```

```
            else:
                try:
                    bank.execute(operation)
                except Exception:
                    pass

    for account_id in sorted(bank.accounts):
        print(account_id, bank.accounts[account_id].balance)


if __name__ == "__main__":
    main()
```

Output :



Q6. Python Module Dependency Resolver

Code :

```
import heapq

def main():
    n, e = map(int, input().split())

    modules = []
```

```
graph = {}
indegree = {}

for _ in range(n):
    module = input().strip()
    modules.append(module)
    graph[module] = set()
    indegree[module] = 0

for _ in range(e):
    a, b = input().split()

    if b not in graph[a]:
        graph[a].add(b)
        indegree[b] += 1

heap = []

for module in modules:
    if indegree[module] == 0:
        heapq.heappush(heap, module)

order = []

while heap:
    module = heapq.heappop(heap)
    order.append(module)

    for dependent in sorted(graph[module]):
        indegree[dependent] -= 1

        if indegree[dependent] == 0:
            heapq.heappush(heap, dependent)

if len(order) == n:
    print(*order)
else:
    print("CYCLE")

    state = {module: 0 for module in modules}
    parent = {}
    cycle = []

    def find_cycle(node):
        state[node] = 1

        for neighbor in graph[node]:
            if state[neighbor] == 0:
                parent[neighbor] = node
```

```
            if find_cycle(neighbor):
                return True

        elif state[neighbor] == 1:
            cycle.append(neighbor)

            current = node
            while current != neighbor:
                cycle.append(current)
                current = parent[current]

            cycle.append(neighbor)
            cycle.reverse()
            return True

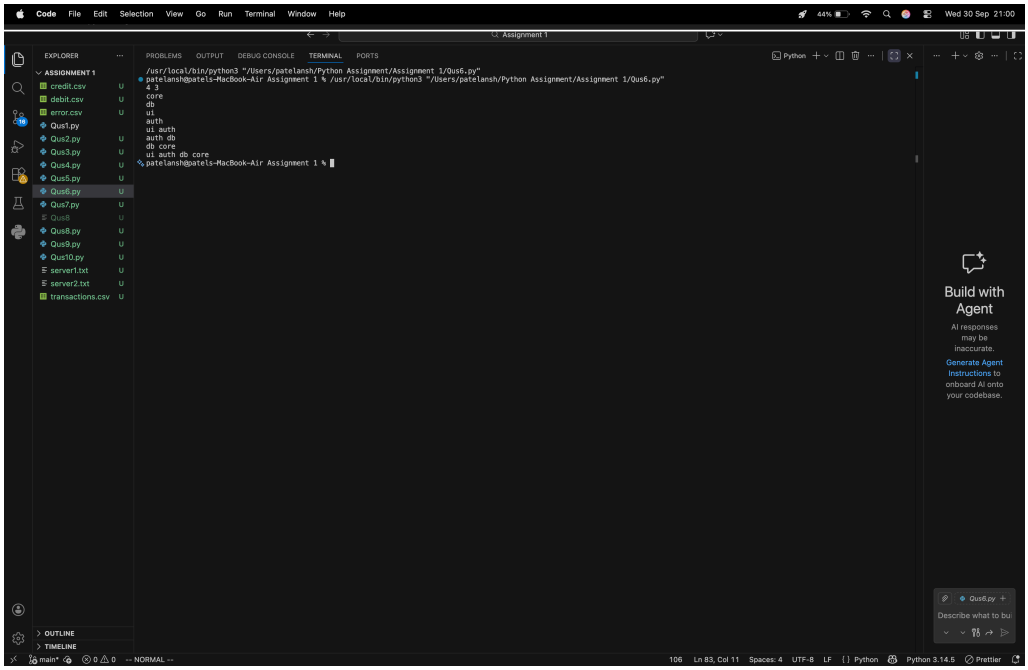
    state[node] = 2
    return False

for module in sorted(modules):
    if state[module] == 0:
        if find_cycle(module):
            break

print(*cycle)


if __name__ == "__main__":
    main()
```

Output :



Q7. Interactive Formula Validator with Custom Exceptions

Code :

```
class InvalidFormatError(Exception):
    pass

class UnknownVariableError(Exception):
    pass

class DivisionByZeroError(Exception):
    pass

class UnsupportedOperatorError(Exception):
    pass


variables = {}

def get_value(value):
    if value in variables:
        return variables[value]

    try:
        return float(value)
    except ValueError:
        raise UnknownVariableError("Unknown variable")


def calculate(left, operator, right):
    a = get_value(left)
    b = get_value(right)

    if operator not in ['+', '-', '*', '/', '%']:
        raise UnsupportedOperatorError("Unsupported operator")

    if operator == '+':
        return a + b
    elif operator == '-':
        return a - b
    elif operator == '*':
        return a * b
    elif operator == '/':
        if b == 0:
            raise DivisionByZeroError("Division by zero")
        return a / b
```

```
    elif operator == '%':
        if b == 0:
            raise DivisionByZeroError("Division by zero")
        return a % b


while True:
    line = input().strip()

    if line.lower() == "quit":
        break

    try:
        if '=' in line:
            parts = line.split('=')

            if len(parts) != 2:
                raise InvalidFormatError("Invalid assignment format")

            name = parts[0].strip()
            value = parts[1].strip()

            if not name.isidentifier():
                raise InvalidFormatError("Invalid variable name")

            variables[name] = get_value(value)

        else:
            parts = line.split()

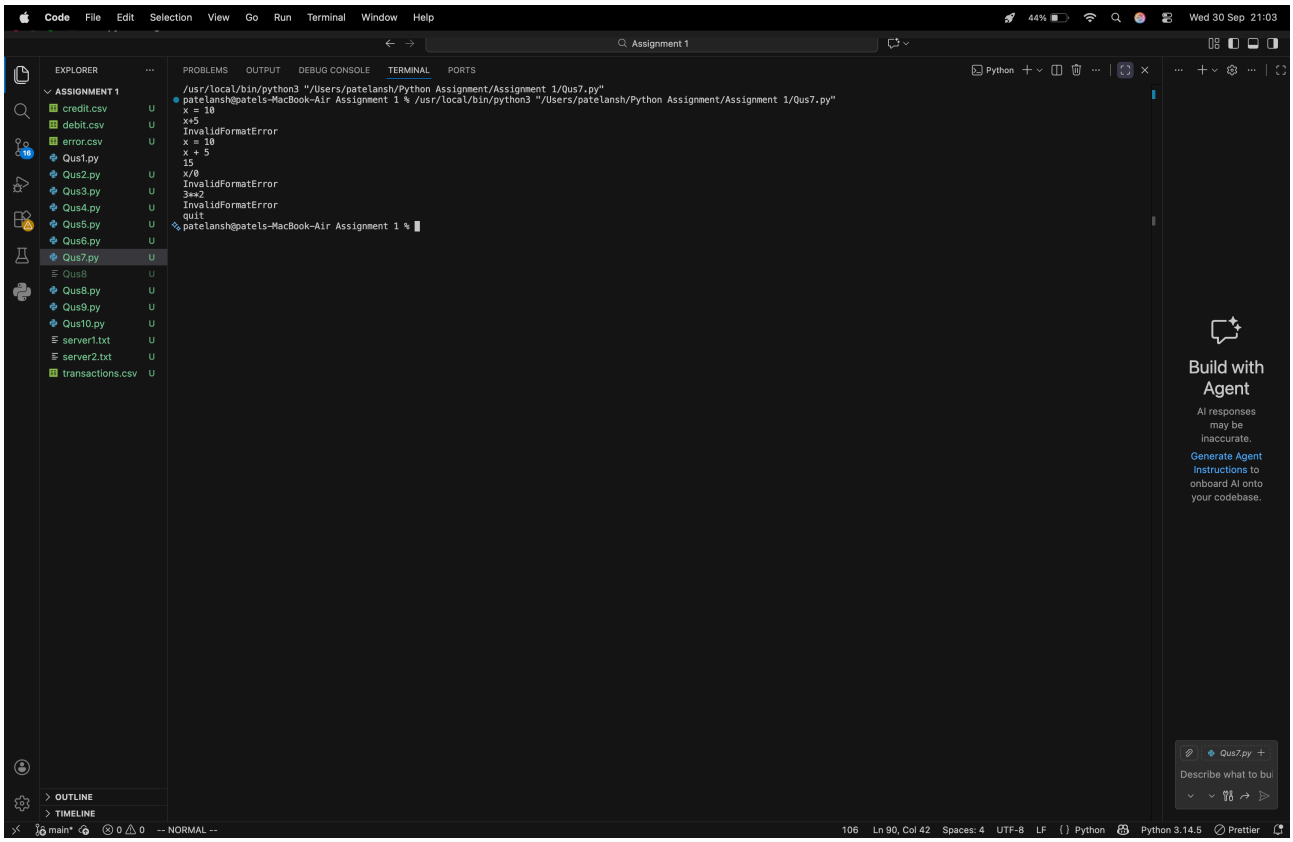
            if len(parts) != 3:
                raise InvalidFormatError("Invalid formula format")

            result = calculate(parts[0], parts[1], parts[2])

            if result.is_integer():
                print(int(result))
            else:
                print(result)

    except InvalidFormatError as e:
        print("InvalidFormatError")
    except UnknownVariableError as e:
        print("UnknownVariableError")
    except DivisionByZeroError as e:
        print("DivisionByZeroError")
    except UnsupportedOperatorError as e:
        print("UnsupportedOperatorError")
```

Output :



Q8. Compressed Log Index using Pickle and Zip

Code :

```
import os
import re
import pickle
import zipfile

folder = input("Enter log folder path: ").strip()

index = {}

for filename in os.listdir(folder):
    filepath = os.path.join(folder, filename)

    if not os.path.isfile(filepath):
        continue
```

```
    if not filename.lower().endswith(".txt"):
        continue

    with open(filepath, "r", encoding="utf-8") as file:
        for line_no, line in enumerate(file, 1):
            tokens = re.findall(r'\b[a-zA-Z0-9]+\b', line.lower())

            for token in set(tokens):
                if token not in index:
                    index[token] = []

                index[token].append((filename, line_no))

index_file = os.path.join(folder, "log_index.pkl")

with open(index_file, "wb") as file:
    pickle.dump(index, file)

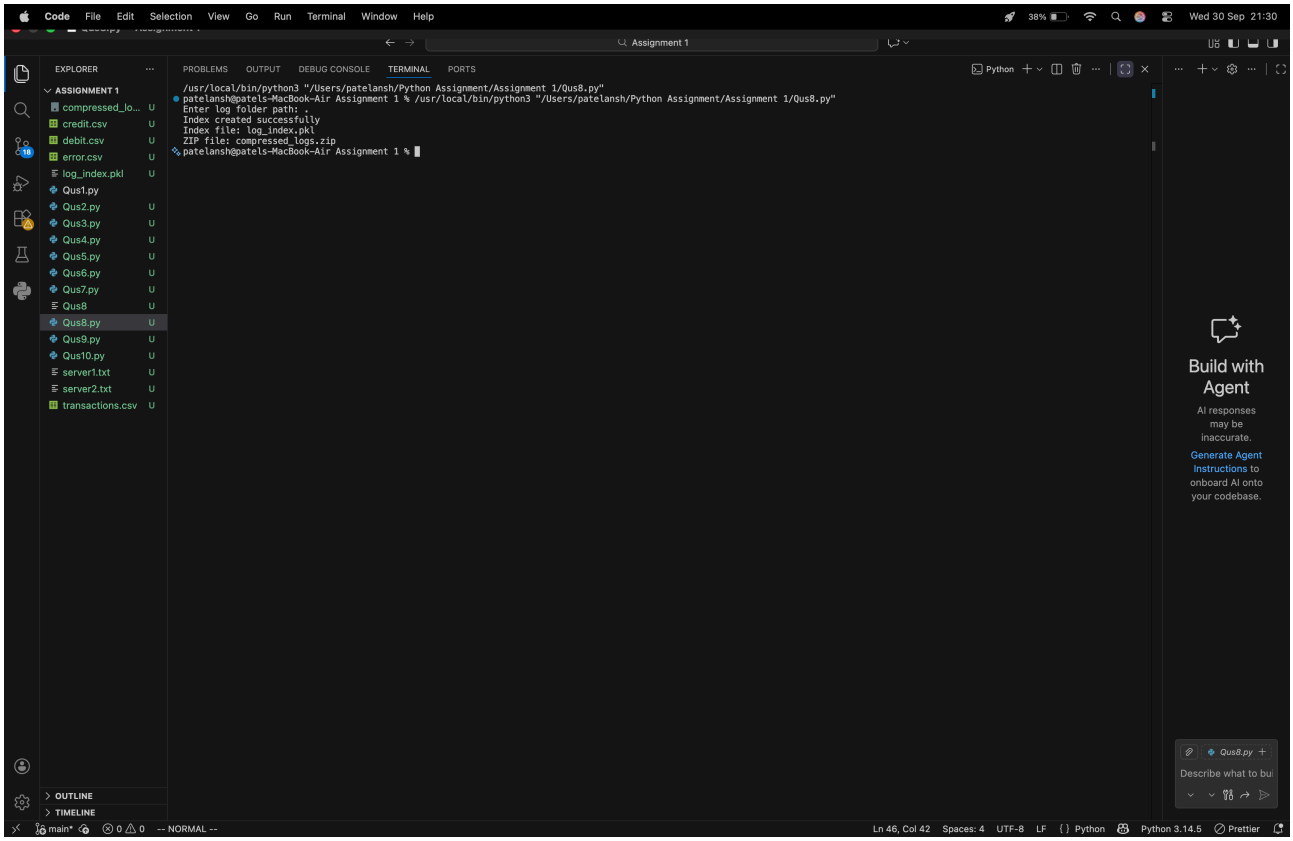
zip_file = os.path.join(folder, "compressed_logs.zip")

with zipfile.ZipFile(zip_file, "w", zipfile.ZIP_DEFLATED) as zipf:
    for filename in os.listdir(folder):
        filepath = os.path.join(folder, filename)

        if os.path.isfile(filepath):
            if filename.endswith(".txt") or filename == "log_index.pkl":
                zipf.write(filepath, filename)

print("Index created successfully")
print("Index file:", "log_index.pkl")
print("ZIP file:", “compressed_logs.zip")
```

Output :



Q9. Threaded Job Scheduler Simulation

Code :

```
import threading
import heapq

class Job:
    def __init__(self, arrival, job_id, priority, duration, resources, order):
        self.arrival = arrival
        self.job_id = job_id
        self.priority = priority
        self.duration = duration
        self.resources = resources
        self.order = order

    def __lt__(self, other):
```

```
                if self.priority != other.priority:
            return self.priority > other.priority
        return self.order < other.order


w, n = map(int, input().split())

jobs = []

for i in range(n):
    arrival, job_id, priority, duration, resources = input().split()

    job = Job(
        int(arrival),
        job_id,
        int(priority),
        int(duration),
        int(resources),
        i
    )

    jobs.append(job)

jobs.sort(key=lambda x: x.arrival)

workers = [0] * w
worker_lock = threading.Lock()
results = []
waiting_times = []


def execute_job(job, worker_id, start_time):
    finish_time = start_time + job.duration

    with worker_lock:
        results.append(
            (job.job_id, worker_id, start_time, finish_time)
        )
        waiting_times.append(start_time - job.arrival)

    return finish_time


current_time = 0
index = 0
queue = []

while index < n or queue:
```

```
    if not queue and index < n:
        current_time = max(current_time, jobs[index].arrival)

    while index < n and jobs[index].arrival <= current_time:
        heapq.heappush(queue, jobs[index])
        index += 1

    available_workers = []

    for i in range(w):
        if workers[i] <= current_time:
            available_workers.append(i)

    while queue and available_workers:
        job = heapq.heappop(queue)
        worker_id = available_workers.pop(0)

        start_time = max(current_time, workers[worker_id])

        thread = threading.Thread(
            target=execute_job,
            args=(job, worker_id + 1, start_time)
        )

        thread.start()
        thread.join()

        workers[worker_id] = start_time + job.duration

    if queue:
        current_time = min(workers)

    elif index < n:
        current_time = max(current_time, jobs[index].arrival)

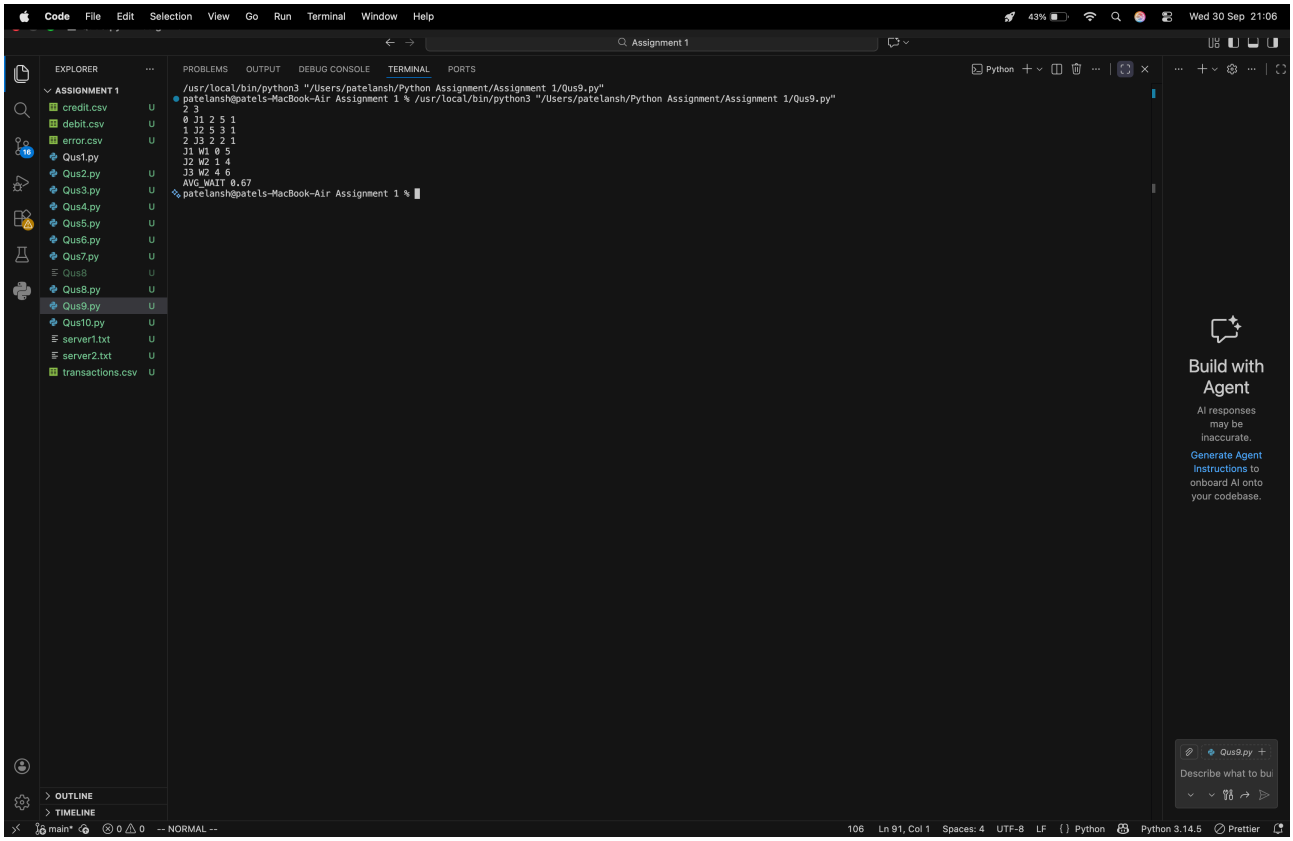

results.sort(key=lambda x: x[2])

for job_id, worker_id, start, finish in results:
    print(job_id, "W" + str(worker_id), start, finish)

average_wait = sum(waiting_times) / len(waiting_times)

print("AVG_WAIT", f"{average_wait:.2f}")
```

Output :



Q10. Tkinter Assignment Tracker with File Persistence

Code :

```
import tkinter as tk
from tkinter import ttk, messagebox
import json
import csv
import os

FILE = "assignments.json"

data = []
```

```
if os.path.exists(FILE):
    with open(FILE, "r") as f:
        data = json.load(f)


def save_data():
    with open(FILE, "w") as f:
        json.dump(data, f, indent=4)


def add_student():
    enrollment = entry_enrollment.get().strip()
    name = entry_name.get().strip()
    assignment = entry_assignment.get().strip()
    status = status_var.get()
    marks = entry_marks.get().strip()
    remarks = entry_remarks.get().strip()

    if not enrollment or not name or not assignment:
        messagebox.showerror("Error", "Please fill all required fields")
        return

    try:
        marks_value = float(marks) if marks else 0
    except ValueError:
        messagebox.showerror("Error", "Marks must be numeric")
        return

    record = {
        "enrollment": enrollment,
        "name": name,
        "assignment": assignment,
        "status": status,
        "marks": marks_value,
        "remarks": remarks
    }

    data.append(record)
    save_data()
    clear_fields()
    display_data()

    messagebox.showinfo("Success", "Record added successfully")


def clear_fields():
    entry_enrollment.delete(0, tk.END)
    entry_name.delete(0, tk.END)
    entry_assignment.delete(0, tk.END)
```

```
    entry_marks.delete(0, tk.END)
    entry_remarks.delete(0, tk.END)
    status_var.set("Pending")


def display_data(records=None):
    for item in tree.get_children():
        tree.delete(item)

    if records is None:
        records = data

    for record in records:
        tree.insert(
            "",
            tk.END,
            values=(
                record["enrollment"],
                record["name"],
                record["assignment"],
                record["status"],
                record["marks"],
                record["remarks"]
            )
        )


def filter_data():
    status = filter_var.get()

    if status == "All":
        display_data()
    else:
        filtered = [
            record for record in data
            if record["status"] == status
        ]
        display_data(filtered)


def export_csv():
    if not data:
        messagebox.showwarning("Warning", "No data available")
        return

    with open("assignment_report.csv", "w", newline="") as f:
        writer = csv.DictWriter(
            f,
            fieldnames=[
```

```
                "enrollment",
                "name",
                "assignment",
                "status",
                "marks",
                "remarks"
            ]
        )

        writer.writeheader()
        writer.writerows(data)

    messagebox.showinfo(
        "Success",
        "CSV report exported as assignment_report.csv"
    )


root = tk.Tk()
root.title("Assignment Tracker")
root.geometry("950x600")

title = tk.Label(
    root,
    text="Student Assignment Tracker",
    font=("Arial", 20, "bold")
)
title.pack(pady=10)

form = tk.Frame(root)
form.pack(pady=10)

tk.Label(form, text="Enrollment").grid(row=0, column=0, padx=5, pady=5)
entry_enrollment = tk.Entry(form)
entry_enrollment.grid(row=0, column=1, padx=5, pady=5)

tk.Label(form, text="Name").grid(row=0, column=2, padx=5, pady=5)
entry_name = tk.Entry(form)
entry_name.grid(row=0, column=3, padx=5, pady=5)

tk.Label(form, text="Assignment").grid(row=1, column=0, padx=5, pady=5)
entry_assignment = tk.Entry(form)
entry_assignment.grid(row=1, column=1, padx=5, pady=5)

tk.Label(form, text="Status").grid(row=1, column=2, padx=5, pady=5)

status_var = tk.StringVar(value="Pending")
status_box = ttk.Combobox(
    form,
```

```
    textvariable=status_var,
    values=["Pending", "Completed"],
    state="readonly"
)
status_box.grid(row=1, column=3, padx=5, pady=5)
tk.Label(form, text="Marks").grid(row=2, column=0, padx=5, pady=5)
entry_marks = tk.Entry(form)
entry_marks.grid(row=2, column=1, padx=5, pady=5)

tk.Label(form, text="Remarks").grid(row=2, column=2, padx=5, pady=5)
entry_remarks = tk.Entry(form)
entry_remarks.grid(row=2, column=3, padx=5, pady=5)
button_frame = tk.Frame(root)
button_frame.pack(pady=10)

tk.Button(
    button_frame,
    text="Add Student",
    command=add_student
).grid(row=0, column=0, padx=5)

tk.Button(
    button_frame,
    text="Clear",
    command=clear_fields
).grid(row=0, column=1, padx=5)

tk.Button(
    button_frame,
    text="Export CSV",
    command=export_csv
).grid(row=0, column=2, padx=5)

tk.Label(button_frame, text="Filter").grid(
    row=0, column=3, padx=5
)

filter_var = tk.StringVar(value="All")

filter_box = ttk.Combobox(
    button_frame,
    textvariable=filter_var,
    values=["All", "Pending", "Completed"],
    state="readonly",
    width=12
)
filter_box.grid(row=0, column=4, padx=5)

tk.Button(
```

```
            button_frame,
    text="Apply",
    command=filter_data
).grid(row=0, column=5, padx=5)

columns = (
    "Enrollment",
    "Name",
    "Assignment",
    "Status",
    "Marks",
    "Remarks"
)
tree = ttk.Treeview(
    root,
    columns=columns,
    show="headings"
)
for column in columns:
    tree.heading(column, text=column)
    tree.column(column, width=140)

tree.pack(
    fill=tk.BOTH,
    expand=True,
    padx=10,
    pady=10
)

display_data()
root.mainloop()
```

Output :

